

Sovereign On-Prem Agentic AI Workbench — Hackathon Build Plan

Constraints: 30 hours, 5+ people, main build machine = friend's GPU laptop

1. What you're actually proving (don't lose sight of this)

Judges aren't grading "did you build MRPL's real system." They're grading whether you can **demo, end-to-end, on one machine, with zero internet calls**:

1. Two+ task types routed to two different local open-weight models automatically
2. One full agentic task (scanned doc → extracted findings → generated Word approval note)
3. One code-generation task executed + verified in a sandbox
4. One multimodal task (image/scanned doc understanding)
5. Visible proof (network monitor / log) that nothing left the machine
6. One AI-automated correspondence task (email draft/send) — kept fully offline via a local mail server, not a cloud email API

Everything else is decoration. If any module is behind at hour 25, cut it — never cut #5, since that's the actual differentiator of the problem statement (a normal RAG chatbot demo won't win this; the *sovereignty proof* will).

2. Offline vs. online — when you actually need internet

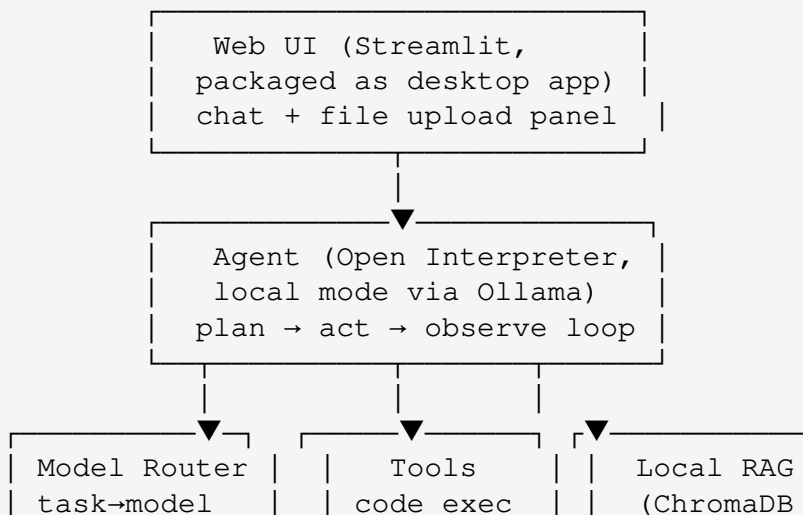
One-time setup only (do this before hackathon day, or in the first hour with wifi):

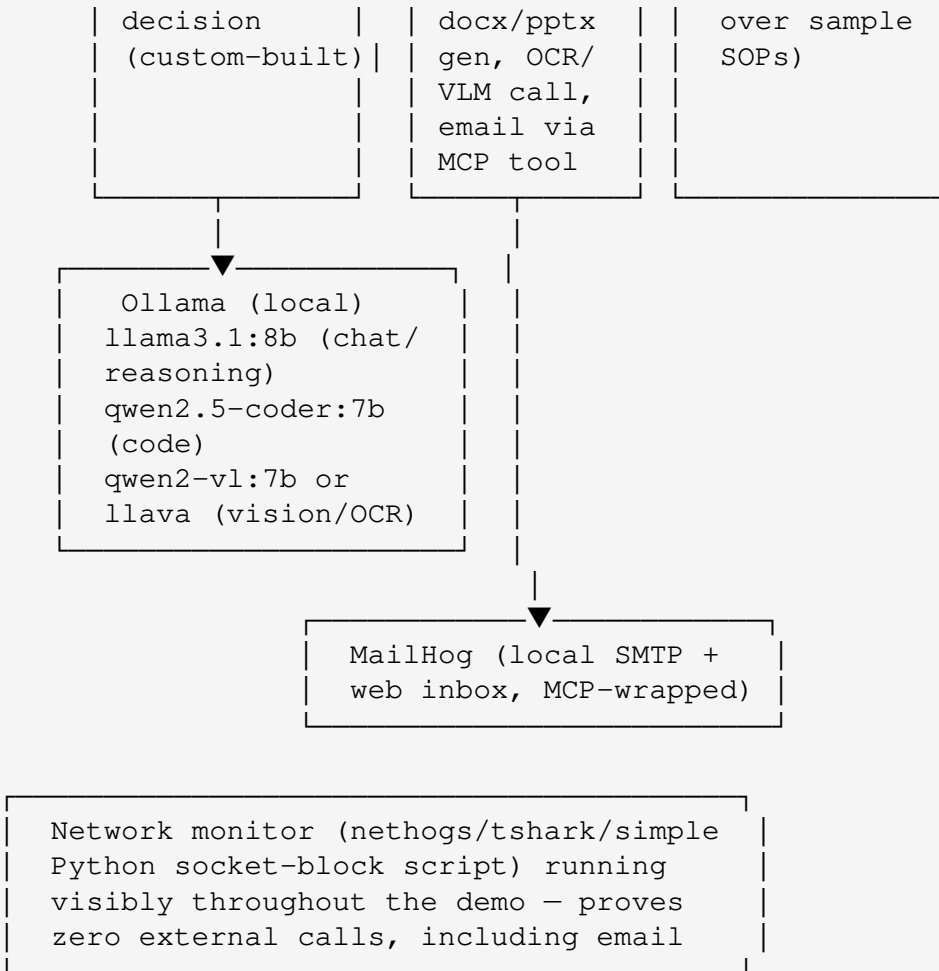
- `ollama pull` for all models
- `pip install` for all Python packages
- Downloading the embedding model for ChromaDB (`sentence-transformers/all-MiniLM`)
- Downloading sample data (SOPs, scanned reports, P&ID images)

Zero internet from that point on — including the entire live demo.

Router, agent loop, OCR, RAG, doc generation, code sandbox, and email (via local mail server) all run entirely on the laptop. Best practice: physically disconnect wifi/ethernet before the live demo and run with the network monitor visible the whole time — proving it beats claiming it.

3. Architecture





4. Build vs. Borrow — don't build everything from scratch

Given the time limit, only build the pieces that ARE the point of the problem statement. Borrow open-source for the rest.

Module	Build it yourselves?	Or use this
Agent loop + code sandbox	No — borrow	Open Interpreter , pointed at local Ollama models (<code>interpreter --local</code>). Confirmed to work fully offline against Ollama/LM Studio. Handles plan→act→observe, code execution, and file read/write out of the box. Run in its "safe mode" / sandboxed, since it has full local shell access by default.
Model routing (task → model)	Yes — build this	This is your differentiator. A small classifier function mapping task type → model name. Judges will ask how this works — know it cold.
RAG / knowledge base	No — borrow	LangChain's or LlamaIndex's Ollama + Chroma integration — a working local RAG pipeline in ~15 lines.
OCR / scanned docs	No — borrow	<code>pytesseract</code> for clean scanned text; <code>unstructured</code> library for messier PDF/image parsing if time allows.
Vision (drawings, handwriting)	No — borrow	Direct API call to the Ollama-hosted vision model (<code>qwen2-vl/llava</code>).
Doc generation	No — borrow	<code>python-docx</code> , <code>python-pptx</code> , <code>openpyxl</code> .
UI	No — borrow	Streamlit (or Chainlit if someone already knows it — shows agent reasoning steps live, good demo visual).

Email automation	Partially build	Local MailHog SMTP server (borrowed) + a thin custom <code>email_tool.py</code> , optionally wrapped as an MCP server (see §7) — the MCP wrapping itself is worth building since it demonstrates the "add capabilities without redesign" requirement.
Network monitoring proof	Yes — build this	Your other differentiator — a simple script or <code>nethogs</code> running visibly. Don't outsource the thing that proves your core claim.

Rule of thumb: anything a judge could reasonably ask "so what did *you* build?" about (router, network-proof, MCP email wrapper) needs to be something you can explain line-by-line. Everything else is legitimate infrastructure reuse.

5. Tech stack (all free, all local)

Piece	Tool	Why
Model serving	Ollama	Trivial multi-model hosting, pull-and-run
Models	<code>llama3.1:8b</code> (general/reasoning), <code>qwen2.5-coder:7b</code> (code), <code>qwen2-vl:7b</code> or <code>llava:7b</code> (vision/OCR)	Fit on a single consumer GPU (8–16GB VRAM) quantized
Agent loop	Open Interpreter , local mode via Ollama	Working agent + code-execution loop out of the box, confirmed offline-capable

RAG	ChromaDB + sentence-transformers (all-MiniLM, local), via LangChain/ LlamaIndex	Zero external calls, minimal code to write
OCR	Tesseract (pytesseract), unstructured as needed	Reliable for clean text; VLM reserved for handwriting/drawings
Sandbox	Open Interpreter's own execution environment, or subprocess with timeout as fallback	Avoid Docker under time pressure unless already known
Doc generation	python-docx, python-pptx, openpyxl	Matches "real deliverables" requirement
Email (offline)	MailHog (local SMTP + web inbox), smtplib, optionally MCP-wrapped	Fully offline automated correspondence — see §7
UI	Streamlit	Fastest usable chat+upload UI
Desktop packaging	pywebview + PyInstaller (optional, polish only)	Wraps the Streamlit app in a native window for presentation — see §8
Network proof	nethogs or a Python socket-monitor script	Core differentiator — don't skip

6. Team split (5+ people)

Role	Owns	Deliverable
------	------	-------------

A — Model & Router	Ollama setup, pulling models, writing the task-classifier	<code>router.py</code> : task/prompt → which model to call
B — Agent Orchestrator	Configuring Open Interpreter against local Ollama, tool wiring	Working <code>interpreter</code> config + tool registration (code exec, file I/O)
C — OCR/ Vision Pipeline	Scanned PDF → text/findings extraction, VLM integration	<code>vision_pipeline.py</code> : PDF/image in → structured findings out
D — RAG + Doc Generation	Chroma ingestion of sample SOPs, retrieval, Word/ PPT generation	<code>rag.py</code> + <code>doc_writer.py</code>
E — UI + Network Proof + Email/MCP + Integration	Streamlit front end, network monitor, MailHog + MCP email tool, wiring everything together, desktop packaging	<code>app.py</code> , <code>network_monitor.py</code> , <code>email_tool.py</code> (MCP-wrapped), final integration, <code>.exe</code> packaging

If you have a 6th person, put them on **testing + the pitch deck/demo narrative** starting hour 12.

7. Email automation module (offline-safe)

Why not a real email API: Gmail/Outlook/MCP connectors require an outbound call to a cloud mail server — this would break the zero-external-calls proof live, on stage, right after claiming it. Skip any cloud email connector entirely.

What to build instead:

1. Run **MailHog** locally (`docker run` or native binary) — gives you a local SMTP server on `localhost:1025` and a web inbox at `localhost:8025`, no internet involved.
2. Write `email_tool.py` using Python's built-in `smtplib` to send through MailHog.
3. Register it as a tool the agent can call — e.g., after drafting an approval note, the agent also drafts and sends a notification email to the relevant team, visible arriving in MailHog's web inbox live during the demo.
4. **Stretch goal (if ahead of schedule):** wrap `email_tool.py` as a small standalone **MCP server** exposing `send_email` as an MCP tool, and have the agent call it over MCP instead of a direct function call. This lets you claim: *"every capability is a swappable MCP tool — adding a new one later means standing up a new MCP server, not redesigning the agent,"* which directly answers the "extensible without redesign" requirement.

This maps to how MRPL would deploy for real too — employees' email would run through the company's own internal mail server, not Gmail, so this isn't a demo hack, it's architecturally accurate.

8. Desktop packaging (optional polish — do this LAST)

Only attempt after all core demo flows work reliably. Adds presentation polish, not functional points.

1. `launcher.py`: starts the Streamlit app as a background subprocess, then opens a `pywebview` native window pointing at `localhost:8501`.
2. Test `launcher.py` directly with Python first before touching PyInstaller.
3. Build: `pyinstaller --onefile --add-data "app.py;." launcher.py`

4. Test the `.exe` on the **same** GPU laptop you'll demo on — PyInstaller builds aren't always portable.
5. Keep a fallback: a `.bat` shortcut running `streamlit run app.py` in case the `.exe` has issues on demo day.
6. Ollama keeps running as its own separate local service — don't try to bundle multi-GB models into the `.exe`; the app only needs to reach `localhost:11434`.

Fit this into the hour 26–28 slot. If not solid by then, fall back to Streamlit in fullscreen/kiosk mode instead.

9. Folder structure

```
mrpl-sovereign-workbench/
├── app.py                    # Streamlit UI entrypoint
├── launcher.py              # pywebview desktop wrapper (
├── agent_config.py          # Open Interpreter config (lo
├── router.py                # task → model selection (cus
├── vision_pipeline.py       # OCR + VLM for scanned docs/
├── rag.py                   # ChromaDB ingestion + retrie
├── doc_writer.py            # docx/pptx/xlsx generation
├── tools/
│   ├── email_tool.py        # MailHog SMTP sender, option
│   └── search_tool.py       # RAG search as an agent tool
├── mcp_servers/
│   └── email_mcp_server.py  # optional: send_email expose
├── network_monitor.py       # visible proof of zero outbo
├── sample_data/
│   ├── sops/                # sample SOP/manual text for
│   ├── scanned_reports/    # sample scanned inspection
│   └── pids/                # sample P&ID images
├── demo_script.md           # exact click-by-click demo f
└── requirements.txt
```

10. Hour-by-hour milestones (30 hrs)

Hours	Milestone	Checkpoint
0–2	Env setup: Ollama installed, all 3 models pulled, Open Interpreter installed and pointed at Ollama, MailHog running, repo scaffolded, roles confirmed	<code>ollama run llama3.1:8b</code> works; <code>interpreter --local</code> responds; MailHog web inbox loads at <code>localhost:8025</code>
2–6	Each owner builds their module standalone	A: router returns correct model for 3 sample tasks. B: Open Interpreter executes a sample code task locally. C: OCR extracts text from 1 sample scan. D: Chroma returns relevant chunk; docx generator makes a sample file. E: Streamlit shows a basic chat box; network monitor logs; MailHog receives a test email via <code>smtplib</code> .
6–12	Modules reach "works alone, end to end for one example"	Each person can demo their own piece in isolation
12–16	First integration checkpoint — wire router + agent + one tool together	Agent takes a request, picks a model via router, calls at least one real tool
16–20	Integrate remaining tools (OCR, RAG, doc-gen, email) into the agent loop	Full pipeline: scanned report → agent reads it → generates approval note as .docx → drafts/sends notification email via MailHog

20–24	Add the coding task path + the multimodal (VLM) path	Agent runs a code task via Open Interpreter's sandbox; describes/extracts from an image via VLM
24–26	Freeze features. No new functionality — bugs only	Full run-through of all demo tasks back to back, wifi off
26–28	Polish UI, desktop packaging (if time allows), write <code>demo_script.md</code> , rehearse pitch	Two full dry-run demos, timed
28–30	Buffer for last-minute breakage + final rehearsal	Nothing new touched — just stability

Hard rule: if integration (hour 12–16 checkpoint) isn't hit, cut the coding-sandbox task or the VLM task first — keep the OCR→approval-note flow, the network proof, and the email module, since together they tell the core story.

11. What "done" looks like for the demo

1. Wifi/ethernet physically disconnected before the demo starts.
2. Judge asks for a code fix → routed to `qwen2.5-coder`, executed via Open Interpreter's sandbox, output shown.
3. Judge hands a sample scanned inspection report → agent OCRs it, extracts findings, retrieves relevant SOP context via RAG, drafts a Word approval note, and drafts/sends a notification email that appears live in MailHog's inbox.
4. Judge hands a P&ID image → VLM describes/extracts info from it.
5. Throughout, a visible terminal/network monitor shows **zero outbound connections** — including during the email send.
6. Explain in 30 seconds: "new models are added by running `ollama pull <model>` and one line in `router.py`; new capabilities are added as new MCP tools — like our email module

— without touching the agent itself."

12. Risks & fallbacks

Risk	Fallback
VLM too slow/ inaccurate on laptop GPU	Pre-select 1–2 sample images that work reliably; avoid live arbitrary judge-provided images unless confident
Open Interpreter's shell access misbehaves	Run in safe/sandboxed mode; cap iterations; have a scripted fallback demo recording as backup
MailHog/email module not ready in time	Cut it — it's additive, not core to proving sovereignty (router + network monitor are)
Docker/sandbox setup eats time	Rely on Open Interpreter's own execution env, or <code>subprocess</code> with a timeout
Model too slow for live demo	Use quantized (Q4) GGUF via Ollama; keep a smaller backup model (e.g., <code>phi3:mini</code>)
Team desyncs on integration	Freeze interfaces early (hour 2–3): agree on function signatures before deep coding
<code>.exe</code> packaging breaks late	Fall back to Streamlit fullscreen/kiosk mode — don't let packaging be a single point of failure

13. If you have time left over

- Add a second knowledge-base source (past correspondence sample) to show multi-source RAG
- Add a simple eval: log which model handled which task, show a small "routing accuracy" table

- Add LoRA/quantization talking points in the pitch even if not implemented
- Wrap additional tools (RAG search, doc-gen) as MCP servers too, to make the "extensible via MCP" story even stronger